

Hybrid Embedding Framework for Memory-Efficient Recommendation Systems

Seung Jin Yang
Seoul National University
Seoul, Republic of Korea
rkem1542@snu.ac.kr

Hyuk-Jae Lee
Seoul National University
Seoul, Republic of Korea
hyuk_jae_lee@capp.snu.ac.kr

Chae Eun Rhee
Hanyang University
Seoul, Republic of Korea
crhee@hanyang.ac.kr

Abstract—This study introduces a memory-efficient mixed representation for deep learning recommendation models (DLRM), addressing the embedding table memory bottleneck from growing data scale. By distinguishing between frequently accessed (hot) and infrequently accessed (cold) embeddings, we store hot embeddings in a compact table while representing cold embeddings using a deep hash embedding (DHE) network, significantly reducing memory usage. This hybrid approach performs table lookups for hot embeddings and parallelized computations for cold embeddings, minimizing training time while maintaining accuracy. Experimental results demonstrate that our method outperforms other embedding reduction techniques in memory efficiency, accuracy, and training speed in CPU-GPU hybrid environments.

I. INTRODUCTION

Recommendation systems are widely used in various fields such as social networks, e-commerce platforms, online streaming services, and digital marketing [1], [2]. These systems analyze user behavior data, such as search history, purchasing patterns, and content preferences, to provide personalized recommendations and support decision-making. Meta's Deep Learning Recommendation Model (DLRM) [3] is one of the most widely used models, converting various categorical data into dense representations and storing them in an embedding table. As datasets continue to grow, more storage space is required for the embedding table to accommodate this increase [4], [5]. In industrial-scale implementations, the size of the embedding table can reach terabytes [6], [7], [8]. DLRM consists of four main operations: the bottom multi-layer perceptron (MLP), embedding lookup, feature interaction, and the top MLP. While the MLP is a compute-dominated task that can easily be accelerated using high-performance GPUs, the embedding lookup, which heavily depends on memory bandwidth, constitutes the largest portion of the total execution time due to memory bottlenecks [4], [9].

Vectors stored in the embedding table can be categorized into hot embeddings and cold embeddings based on their access frequency. While the threshold may vary depending on the context, hot embeddings are generally defined as the vectors that account for more than 80% of the total access frequency [10]. These hot and cold embeddings follow a long-tail distribution [11], [8], where hot embeddings, which account for most memory accesses, represent less than 10% of the entire dataset. On the other hand, the remaining 90% or

more consists of cold embeddings [12]. In experiments with the Criteo Kaggle dataset, it was found that hot embeddings, which accounted for 80% of the total accesses, made up only 6.2% of the entire embedding [12]. Cold embeddings, while having a lower access frequency and minimal impact on model accuracy [13], represent a significant portion of the overall memory demand [11].

In this study, we propose a hybrid embedding framework that reflects the characteristics of hot and cold embedding vectors. Hot vectors, which have high access frequency, are stored directly in memory, while large-scale cold embedding vectors are replaced by neural representations. Instead of directly querying cold vectors from memory, they are computed through a compact deep hash embedding (DHE) network, effectively reducing memory requirements. The key benefits of replacing cold data with the DHE network are twofold: first, it allows for GPU acceleration, and second, the level of approximation can be flexibly adjusted through network design. However, the process of computing cold vectors using the DHE network takes more time than direct memory lookup, which introduces a trade-off between memory savings and increased computation time. To address this, the system handles hot vector retrieval and cold vector computation in parallel, minimizing execution time overhead. Compared to the traditional approach of processing all embedding data through table lookups, the proposed method reduces memory requirements by approximately 5%, while maintaining similar execution time and accuracy.

II. BACKGROUND

A. Related Work

Research aimed at reducing the storage space of embedding vectors and accelerating lookup operations in DLRM has been ongoing, and these embedding compression techniques can be broadly classified into three categories: low-precision [14], [15], mixed-dimension [13], [16], [17], and hashing [18], [19] techniques. The low-precision method reduces memory usage and increases computational speed by representing embedding vectors, originally stored in 32-bit floating-point format, with fewer bits. However, it comes with the downside of accuracy degradation. The mixed-dimension method, unlike the traditional approach where all vectors are assigned the same dimension, adjusts dimensions based on data usage

frequency [13] or creates sparse embedding tables through pruning [16], [17]. This method can solve overfitting problems or provide high accuracy, but may incur additional costs in the training and inference process, making it inefficient. Hashing tricks [18] and compositional embedding [19] map large-dimensional token spaces to smaller dimensions using hash functions. While using a naive hash function can reduce memory requirements, it can lead to accuracy loss. To improve accuracy, the quotient-remainder trick can be used to divide the embedding table into two separate tables and combine the results, but this requires additional memory access and computation.

Conventional embedding compression techniques have had limitations in simultaneously reducing table size and maintaining model accuracy. To overcome this, a method was proposed where data is processed through computation instead of lookup. The DHE Network [20] is a model that combines hash embedding with an embedding network. It transforms the input index into a unique identifier through dense hash encoding, and then uses a deep embedding network to generate feature embeddings. DHE has been shown to achieve similar accuracy with approximately a quarter of the memory cost compared to models using traditional embedding tables, as demonstrated on the Movielens-20M [21] dataset.

Research that takes an architectural approach, rather than an algorithmic one, has proposed optimizing memory access performance by storing hot and cold embedding vectors in separate spaces. Frequently accessed hot vectors are placed in nearby high-speed memory, while infrequently used cold vectors are placed in remote memory, thereby improving overall memory performance [22], [23].

B. Problem Statements

In various studies aimed at reducing the storage space of embedding tables and shortening embedding lookup times, it has been found that deterministic methods cannot solve all the problems, and learning-based approaches introduce additional overhead. In particular, learning-based methods [19], [13], [16], [20] improve memory usage and model performance by utilizing adaptive hyperparameters that reflect the characteristics of embedding data. However, these methods are highly sensitive to the setting of hyperparameters, which can significantly affect performance. Additionally, when there is insufficient memory budget and the number of model parameters is inadequate, the accuracy can drop significantly compared to the original DLRM embedding approach.

The hybrid data embedding (HDE) approach proposed in this paper applies the DHE network only to cold data, thereby reducing memory requirements and minimizing accuracy loss due to approximation. While the proposed method of processing cold data with the DHE network involves a trade-off between reduced memory usage and increased execution time, it offers several key differentiators. The DHE network dedicated to cold data can be reduced in size, allowing for more aggressive approximations. Additionally, the DHE network can be executed in parallel with the table lookup

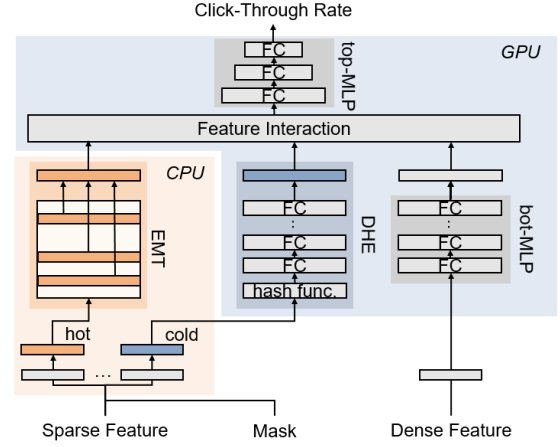
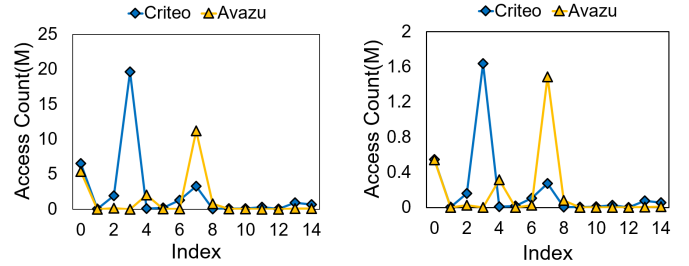


Fig. 1: Overall architecture of our proposed method, Hybrid Data Embedding(HDE).



(a) access count for train data (b) access count for test data

Fig. 2: Access count profiling on train/test data. The first 15 indices of the embedding table are used for profiling. The first and the third embedding table of Criteo and Avazu are used respectively.

of hot embedding data. When querying multiple embedding tables, hot and cold data are mixed, but by implementing cold embedding computations in parallel with the hot embedding table lookups, the computation time of the DHE network is hidden behind the table lookup time, maximizing efficiency. The proposed approach maintains model performance even with a small number of parameters, and it has the advantage of being less reliant on hyperparameters for performance.

III. PROPOSED METHODS

A. Architecture Overview

Figure 1 shows the overall architecture of the proposed model, which follows a structure similar to the CPU-GPU hybrid architecture that utilizes both CPU and GPU [24], [12]. The model takes sparse features, dense features, and a mask as inputs. The mask contains information indicating whether the embedding vector referenced by the sparse feature is hot or cold. Based on the access count of the training data, the top 5% are classified as hot, while the remaining data is classified as cold. Figures 3(a) and (b) show the access count analysis of the Criteo [25] and Avazu [25] datasets, respectively, indicating that the access ratio of embedding data with the same index is similar in both the training and testing data. This suggests that data classified as hot in training can be assumed to be hot in testing as well. The sparse features and dense features

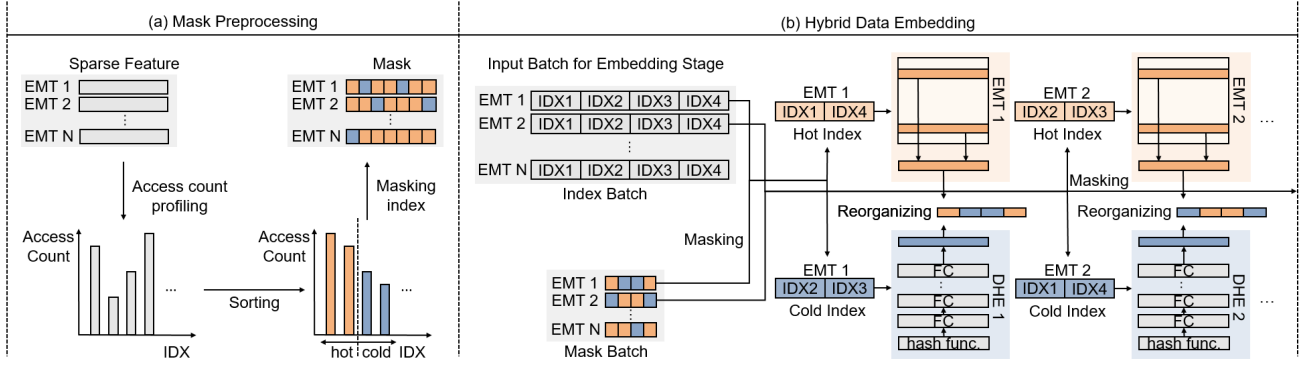


Fig. 3: Overall process of the hybrid data embedding.

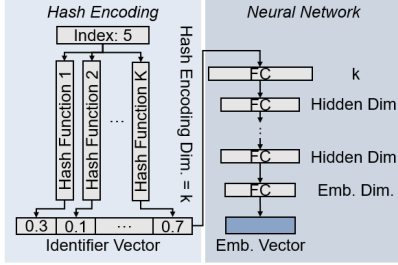


Fig. 4: DHE network computation. The number of the hash functions for hash encoding is equal to the hash encoding dimension k .

are processed through different paths. Dense features, like in conventional systems, are processed through the bottom MLP layer on the GPU. On the other hand, sparse features are divided into hot and cold features on the CPU via the mask. Hot features are processed through embedding table lookups on the CPU, while cold features are computed using the DHE network on the GPU. These two embedding vectors are concatenated, and their results, along with the bottom MLP layer's output, are passed through feature interaction before going through the top MLP layer to produce the final Click-Through Rate (CTR). Section III.C shows how embedding table and network aggregation techniques are employed to optimize the parallel processing of embedding lookups and DHE operations.

B. Hybrid Data Embedding: Embedding Tables and DHE for Efficient Data Handling

The overall process of the proposed HDE is depicted in Figure 3. Let's assume an example where the batch size is 4 and there are N embedding tables. Before applying HDE, a preprocessing step for the mask is required. In the mask preprocessing stage shown in Figure 3 (a), the access count for each embedding table is first analyzed based on the sparse features. The indices are then sorted according to these access counts, and the top 5% of indices are classified as hot, while the remaining indices are classified as cold, resulting in the creation of the mask. Figure 3 (b) shows the HDE stage. The input batch for HDE consists of an index batch and a pre-

generated mask batch. First, the indices for the first embedding table in the index batch are divided into hot and cold indices using the mask batch. For example, the indices $IDX1$, $IDX2$, $IDX3$, and $IDX4$ for the first embedding table are classified into hot ($IDX1$ and $IDX4$) and cold ($IDX2$ and $IDX3$) based on the mask batch. Next, the hot indices are used to lookup the embedding table to retrieve the embedding vectors, while the cold indices are processed through the first DHE network to compute the embedding vectors. Finally, the output embedding vectors for each type of index (hot and cold) are rearranged in the original batch order, and the same procedure is repeated for the next embedding table.

Figure 4 illustrates the process of calculating cold embedding vectors using the proposed DHE network, which follows a structure similar to a typical DHE network [20]. The DHE network consists of two main components: hash encoding and a neural network. First, the cold index is input into K hash functions, which generate a K -dimensional identifier vector. Here, K represents the hash encoding dimension. The generated identifier vector is then used as the input to a neural network consisting of fully connected (FC) layers to compute the embedding vector. Compared to a typical embedding table lookup, the identifier vector corresponds to the row index of the embedding table, and the neural network takes over the role of the embedding table. The DHE network can progressively approximate the ground-truth embedding vector through training. In contrast to traditional embedding table lookup, which requires storing a row for each item in the embedding table and thus demands large memory, the DHE network implicitly represents data through the neural network, significantly reducing memory requirements.

Since embedding vector computation and embedding table lookup are independent tasks, the table lookup for hot embedding vectors can be processed on the CPU, while the computation of cold embedding vectors through the DHE network can be handled on the GPU in parallel. Through this hybrid data embedding approach, cold embedding vectors, which account for about 90% or over of the traditional embedding table, can be replaced with much smaller DHE network parameters. This results in generating embedding vectors with only about 5% of the memory required by the traditional

embedding table, significantly reducing memory usage. Unlike the traditional DHE network, the proposed HDE approach does not replace the entire embedding table. Instead, only the cold embedding vectors are replaced with the DHE network, thus reducing memory usage more aggressively while minimizing performance degradation. This is because cold embedding vectors occupy only a very small portion of the total access count, and their impact on the overall model performance is minimal [13].

C. HDE Optimization Through Table and Network Aggregation

Through HDE, embedding lookup and computation can be performed in parallel, but several issues can arise with a naive implementation shown in Figure 3 (b). First, significant time overhead occurs in the process of distinguishing hot and cold indices and reorganizing the results. If there are N embedding tables, the task of distinguishing hot and cold indices using the mask and the task of reorganizing the embedding table lookup results for hot indices and the embedding vector computation results for cold indices will be repeated N times. Second, the DHE network replaces the embedding table, meaning that the number of DHE networks needed is equal to the number of embedding tables. As the number of embedding tables increases, the number of parameters for the DHE network also increases, leading to higher computational costs due to repeated embedding vector computations. Even though the embedding vector computation is performed in parallel with the embedding table lookup, if the computation time is too long, the lookup time may not be able to hide it. Lastly, the embedding table lookup for hot indices and the embedding vector computation for cold indices can cause bottlenecks. For example, if the lookup for hot indices in the first embedding table completes quickly, the process must wait for the cold index vector computation to finish, which could delay the lookup for the second embedding table.

Figure 5 shows the improvement of parallel processing for embedding lookup and computation from Figure 3 (b). Before proceeding with embedding lookup and computation, the indices for all embedding tables in the input batch are masked. As shown in Figure 4 (a), the first embedding table's mask identifies $IDX1$ and $IDX4$ as hot indices for EMT 1, and the second embedding table's mask identifies $IDX2$ and $IDX3$ as hot indices for EMT 2. Cold indices are masked in the same way. The hot and cold indices, once classified, are gathered separately. Hot indices are looked up in the aggregated embedding table all at once, and cold indices are computed all at once using a single DHE network. Figure 5 (b) shows the embedding table lookup using the aggregated embedding table in more detail. The aggregated embedding table is created by concatenating multiple embedding tables into one large table. As a result, preprocessing of the hot indices for the aggregated embedding table is necessary. The new index for a given $IDX i$ in the original n -th embedding table can be calculated by adding the row sizes of the first embedding table through the $(n-1)$ -th embedding table, and then adding $IDX i$. Depending

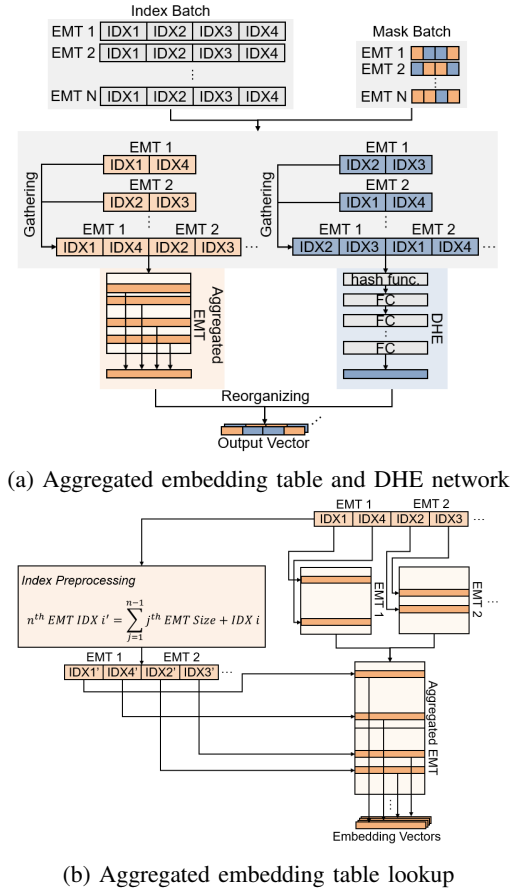


Fig. 5: Aggregated hybrid data embedding

on the dataset, the number of items in each embedding table can vary greatly. For smaller embedding tables, there may be only 3-4 items, while larger embedding tables can have more than 10 million items. Smaller embedding tables can be entirely stored in memory without significantly changing the overall memory requirements. Since the DHE network is intended to reduce the data size by investing in processing time, replacing already small embedding tables with the DHE network would be inefficient. Therefore, during preprocessing of the mask, only embedding tables with a large number of items are processed with the hot/cold mask for HDE, reducing unnecessary computation. The method described above solves the hot/cold overhead and computational cost issues that arise in a naive approach by performing the masking and embedding vector computation only once. Additionally, by masking and gathering all cold and hot indices in the batch before computation, it resolves the bottleneck problem that existed between embedding table lookup and embedding vector computation.

IV. EXPERIMENTS

This section describes the experimental setup, including the datasets used, evaluation metrics, and implementation details. Subsequently, the effectiveness of the proposed HDE method is evaluated through comparisons with existing embedding reduction techniques.

TABLE I: Overall Results. The best and second-best results are **bolded** and underlined

Embedding Methods	Criteo					Avazu				
	Memory Reduction	AUC	Accuracy(%)	Logloss	Exe. Time(s)	Memory Reduction	AUC	Accuracy(%)	Logloss	Exe. Time(s)
FDE	1	0.8022	<u>78.609</u>	0.4477	323.74	1	<u>0.7493</u>	84.551	0.3927	141.92
DHE	0.268	0.7662	77.156	0.4765	4305.18	0.257	0.7280	84.367	0.4048	2927.91
CE	<u>0.250</u>	0.7931	78.359	0.4556	632.31	<u>0.250</u>	0.7460	84.531	0.3968	230.72
HDE	0.051	<u>0.8009</u>	78.612	<u>0.4491</u>	316.50	0.050	0.7516	84.577	<u>0.3934</u>	<u>146.25</u>

A. Experimental Setup

Datasets We conducted experiments using two widely used CTR prediction datasets, Criteo [25] and Avazu [25]. The Criteo dataset consists of advertisement click records from 45 million users over a 7-day period and includes 13 dense features and 26 categorical features. The data from the first 6 days was used as the training set, and the data from the last 7th day was split into validation and test sets in a 1:1 ratio. The Avazu dataset consists of mobile ad click records from an 11-day period and includes 8 dense features and 13 categorical features. 80% of the user-item interactions were used as the training set, with the remaining 20% split equally into validation and test sets.

Comparisons The conventional techniques used for comparison include full-dimension embedding (FDE) [3], compositional embedding (CE) [19], and DHE [20]. FDE is the baseline method used in the original DLRM, where the same embedding dimension is applied across all feature fields. CE utilizes the quotient-remainder trick to replace the embedding table with smaller embedding tables. In the experiments, the QR collision value was set to 4. DHE combines hash encoding and a neural network to replace the embedding table, generating embedding vectors through computation rather than lookup. For the experiments, we set the size of the DHE network to approximately one-quarter of the size of the FDE.

Evaluation metrics For performance comparison, memory reduction, execution time, area under the receiver operating characteristic curve (AUC), and Logloss were used as evaluation metrics. Memory reduction is a metric representing the size of the experimental method relative to the baseline FDE's embedding table size, normalized based on the FDE's embedding table size. Execution time measures the effective latency during actual parallel execution of the task. For the baseline, since embedding lookup and the bottom MLP are performed simultaneously, the execution time is measured as the time until both tasks are completed. AUC represents the probability that a positive instance is ranked higher than a randomly selected negative instance, with a higher value indicating better performance. Logloss is the commonly used binary cross-entropy loss, with a lower value indicating better performance. In DLRM, the embedding table size constitutes the majority of the model parameters [5], and differences of 0.001 in AUC or Logloss are considered significant in CTR prediction tasks [26], [27].

Implement Details In the proposed HDE, hot/cold masking is performed by classifying the top 5% of vectors based on

access count as hot, and the remaining vectors as cold. The hash bucket size for HDE is set to 10^6 , and the hash encoding dimension, hidden layer dimension, and FC layer count are set to 1024, 256, and 9 for the Criteo dataset, respectively, and 128, 128, and 5 for the Avazu dataset. The embedding dimension for the DLRM model used in the experiments is 16, and the top MLP layer's dimension is configured as "512-64-1". The bottom MLP layer is designed considering the dataset size and the number of dense features, with "13-512-256-64-16" for the Criteo dataset and "8-256-64-16" for the Avazu dataset. DLRM training is conducted for 1 epoch, with a batch size set to 128. The learning rate is 0.1 for FDE, DHE, and HDE using stochastic gradient descent (SGD), and 0.01 for CE using adaptive gradient (AdaGrad). The experiments are performed on a single AMD EPYC 7742 64-core processor and a single Nvidia Tesla A100 GPU, and the execution time for each task is profiled using Python's line profiler.

B. Overall Results

In the CPU-GPU hybrid environment, for the baseline FDE, the embedding lookup is performed on the CPU while the bottom MLP is executed on the GPU. For DHE and CE, the embedding lookup is performed on the GPU, while the bottom MLP runs on the CPU. The results for each embedding method are summarized based on the fastest execution time. Table I shows the overall results.

Comparison with baseline When comparing FDE with the proposed HDE, we can see that HDE significantly reduces memory usage, utilizing only 5.1% and 5.0% of FDE's parameters for the Criteo dataset and Avazu dataset, respectively. Specifically, for the Avazu dataset, memory usage is reduced to 5.0% of FDE's memory, while AUC and accuracy improve by about 0.0023 and 0.026%, respectively, and training time remains similar to the baseline.

Comparison with other methods When comparing HDE with other embedding methods, it outperforms in all metrics such as memory reduction, accuracy, and execution time. Compared to DHE, for the Criteo dataset, HDE uses 5.25 times fewer parameters, has 13.60 times faster training time, and shows an accuracy improvement of approximately 1.46%. Compared to CE, for the Criteo dataset, HDE uses about 5 times fewer parameters, has approximately twice as fast training time, and achieves 0.253% higher accuracy.

C. Latency Breakdown

Table II shows the latency breakdown between the baseline FDE and the proposed HDE. While there is data transfer

TABLE II: Latency breakdown on Criteo Kaggle dataset.

Embedding Methods	CPU time (s)			GPU time (s)			Overall time (s) (CPU/GPU)
	Masking	Index Preprocessing	Embedding Lookup	Bot MLP	DHE Computation	Reorganizing	
FDE	-	-	323.74	77.73	-	-	323.74/77.73
HDE	79.14	89.28	90.96	77.73	218.39	20.38	259.38/316.50

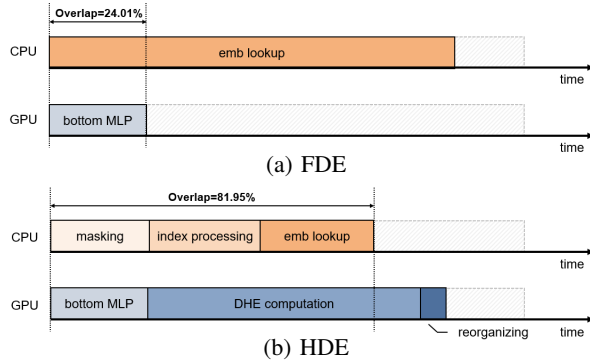


Fig. 6: Timing diagrams of the (a) full dimension embedding(FDE) and (b) the proposed hybrid data embedding(HDE). In the diagram, the length of each task is proportional to the actual latency.

from the CPU to the GPU in HDE, the data transfer latency is negligible when considering the GPU bandwidth and the size of the transferred data. As a result, it was not reported separately. According to the experimental results, FDE takes 323.74s for embedding lookup on the CPU and 77.73s for the bottom MLP on the GPU, with about 24.01% overlap between the two. In contrast, HDE takes 259.38s for embedding lookup on the CPU and 316.50s for the bottom MLP on the GPU, with about 81.95% overlap. The high overlap percentage indicates that resources are being utilized more efficiently during execution [28].

Figure 6 shows the timing diagram when embedding lookup and bottom MLP are performed in parallel in the CPU-GPU hybrid environment. In Figure 6(a), for FDE, it is observed that there is significant idle time where the GPU waits for the CPU’s embedding lookup to finish after completing the bottom MLP. On the other hand, in Figure 6(b) for HDE, we see that the work between the CPU and GPU is effectively distributed. While the GPU performs the bottom MLP, the CPU handles masking and index preprocessing, and while the CPU performs the embedding table lookup, the GPU simultaneously performs the DHE computation, increasing the overlap between CPU and GPU operations and thereby efficiently utilizing resources.

D. Analysis of Hybrid Data Embedding

One of the key contributions of HDE is the division of embedding vectors into hot and cold categories based on their access frequency, with only the cold embedding vectors being computed through the DHE network. Figure 7 shows the relationship between the proportion of cold vectors computed by the DHE network, the test AUC, and the number of parameters. In Figure 7(a), as the proportion of cold vectors computed

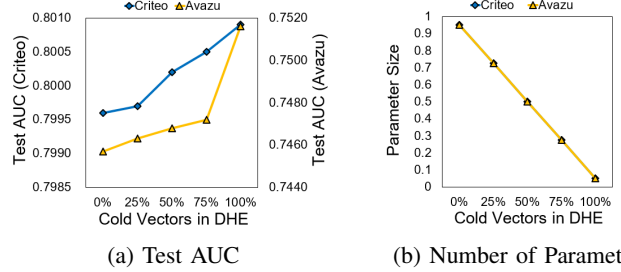


Fig. 7: Test AUC and number of parameters for varying portion of cold vectors in DHE.

by the DHE network increases from 0% to 100%, we observe a gradual increase in test AUC. Additionally, Figure 7(b) shows that as the proportion of cold vectors increases, the total number of parameters decreases. This indicates that processing only the cold embedding vectors through the DHE network based on data *hotness* is an efficient method that can maintain high performance with fewer parameters.

V. CONCLUSION

This study proposed the HDE framework to address the memory bottleneck in DLRMs by storing frequently accessed hot data in embedding tables and computing less frequently accessed cold data using a DHE network. This approach reduced memory usage by approximately 20 times compared to baseline methods while maintaining accuracy and training time, with efficient resource utilization achieved through parallel execution in a CPU-GPU hybrid environment. Future work includes incorporating dynamic hot/cold classification, optimizing DHE network architectures, scaling to multi-GPU systems, and generalizing the framework to other applications, enhancing its scalability and versatility for large-scale recommendation systems.

VI. ACKNOWLEDGMENT

This work was partly supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2020-II201373, Artificial Intelligence Graduate School Program at Hanyang University), the Technology Innovation Program (or Industrial Strategic Technology Development Program) (RS-2023-00235718, 23040-15FC) funded by the Ministry of Trade, Industry & Energy (MOTIE, Korea) (1415187505), and the Information Technology Research Center (ITRC) support program (IITP-2025-2020-0-01461) supervised by IITP and funded by the MSIT.

REFERENCES

- [1] Hang Xue and Dongmei Zhang. A recommendation model based on content and social network. In *2019 IEEE 8th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*, pages 477–481, 2019.
- [2] Doae Mensouri and Abdellah Azmani. A new marketing recommendation system using a hybrid approach to generate smart offers. *Applied Computer Systems*, 27(2):149–158, 2022.
- [3] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Malleevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. Deep learning recommendation model for personalization and recommendation systems, 2019.
- [4] Bilge Acun, Matthew Murphy, Xiaodong Wang, Jade Nie, Carole-Jean Wu, and Kim Hazelwood. Understanding training efficiency of deep learning recommendation models at scale. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 802–814, 2021.
- [5] Ehsan K. Ardestani, Changkyu Kim, Seung Jae Lee, Luoshang Pan, Jens Axboe, Valmiki Ramprasad, Banit Agrawal, Fuxun Yu, Ansha Yu, Trung Le, Hector Yuen, Dheevatsa Mudigere, Shishir Juluri, Akshat Nanda, Manoj Wodekar, Krishnakumar Nair, Maxim Naumov, Chris Petersen, Mikhail Smelyanskiy, and Vijay Rao. Supporting massive dlrn inference through software defined memory. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*, pages 302–312, 2022.
- [6] Michael Lui, Yavuz Yetim, Özgür Özkan, Zhuoran Zhao, Shin-Yeh Tsai, Carole-Jean Wu, and Mark Hempstead. Understanding capacity-driven scale-out neural recommendation inference. In *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 162–171, 2021.
- [7] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, et al. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 993–1011, 2022.
- [8] Aditya Desai and Anshumali Shrivastava. The trade-offs of model size in large recommendation models: 100gb to 10mb critero-tb dlrn model. *Advances in Neural Information Processing Systems*, 35:33961–33972, 2022.
- [9] Daochen Zha, Louis Feng, Qiaoyu Tan, Zirui Liu, Kwei-Herng Lai, Bhargav Bhushanam, Yuandong Tian, Arun Kejariwal, and Xia Hu. Dreamshard: generalizable embedding table placement for recommender systems. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2024. Curran Associates Inc.
- [10] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J. Nair. Heterogeneous acceleration pipeline for recommendation system training. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1063–1079, 2024.
- [11] Saurabh Agarwal, Chengpo Yan, Ziyi Zhang, and Shivaram Venkataraman. Bagpipe: Accelerating deep recommendation model training. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP ’23, page 348–363, New York, NY, USA, 2023. Association for Computing Machinery.
- [12] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J. Nair. Accelerating recommendation system training by leveraging popular choices. *Proceedings of the VLDB Endowment*, 15(1):127–140, September 2021.
- [13] A.A. Ginart, Maxim Naumov, Dheevatsa Mudigere, Jiyan Yang, and James Zou. Mixed dimension embeddings with application to memory-efficient recommendation systems. In *2021 IEEE International Symposium on Information Theory (ISIT)*, pages 2786–2791, July 2021.
- [14] Jian Zhang, Jiyan Yang, and Hector Yuen. Training with low-precision embedding tables. In *Systems for Machine Learning Workshop at NeurIPS*, volume 2018, 2018.
- [15] Hui Guan, Andrey Malevich, Jiyan Yang, Jongsoo Park, and Hector Yuen. Post-training 4-bit quantization on embedding tables. *arXiv preprint arXiv:1911.02079*, 2019.
- [16] Zirui Liu, Qingquan Song, Li Li, Soo-Hyun Choi, Rui Chen, and Xia Hu. Pme: pruning-based multi-size embedding for recommender systems. *Frontiers in big Data*, 6:1195742, 2023.
- [17] Wei Deng, Junwei Pan, Tian Zhou, Deguang Kong, Aaron Flores, and Guang Lin. Deeplight: Deep lightweight feature interactions for accelerating ctr predictions in ad serving. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, WSDM ’21, page 922–930, New York, NY, USA, 2021. Association for Computing Machinery.
- [18] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML ’09, page 1113–1120, New York, NY, USA, 2009. Association for Computing Machinery.
- [19] Hao-Jun Michael Shi, Dheevatsa Mudigere, Maxim Naumov, and Jiyan Yang. Compositional embeddings using complementary partitions for memory-efficient recommendation systems. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD ’20, page 165–175, New York, NY, USA, 2020. Association for Computing Machinery.
- [20] Wang-Cheng Kang, Derek Zhiyuan Cheng, Tiansheng Yao, Xinyang Yi, Ting Chen, Lichan Hong, and Ed H. Chi. Learning to embed categorical features without embedding tables for recommendation. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD ’21, page 840–850, New York, NY, USA, 2021. Association for Computing Machinery.
- [21] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.*, 5(4), December 2015.
- [22] Cheng-Yu Chen, Jui-Nan Yen, You-Ru Lai, Yun-Ping Lin, and Chia-Lin Yang. Rects: A temporal-aware memory system optimization for training deep learning recommendation models. In *Proceedings of the 17th ACM International Systems and Storage Conference*, SYSTOR ’24, page 104–117, New York, NY, USA, 2024. Association for Computing Machinery.
- [23] Hiwot Tadese Kassa, Paul Johnson, Jason Akers, Mrinmoy Ghosh, Andrew Tulloch, Dheevatsa Mudigere, Jongsoo Park, Xing Liu, Ronald Dreslinski, and Ehsan K Ardestani. Mtrains: Improving dlrn training efficiency using heterogeneous memories. *arXiv preprint arXiv:2305.01515*, 2023.
- [24] Keshav Balasubramanian, Abdulla Alshabanah, Joshua D Choe, and Murali Annavaram. cdlrm: Look ahead caching for scalable training of recommendation models. In *Proceedings of the 15th ACM Conference on Recommender Systems*, RecSys ’21, page 263–272, New York, NY, USA, 2021. Association for Computing Machinery.
- [25] Jieming Zhu, Jinyang Liu, Shuai Yang, Qi Zhang, and Xiuqiang He. Open benchmarking for click-through rate prediction. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, CIKM ’21, page 2759–2769, New York, NY, USA, 2021. Association for Computing Machinery.
- [26] Gopi Krishna Jha, Anthony Thomas, Nilesh Jain, Sameh Gobriel, Tajana Rosing, and Ravi Iyer. Mem-Rec: Memory efficient recommendation system using alternative representation. In Berrin Yanikoğlu and Wray Buntine, editors, *Proceedings of the 15th Asian Conference on Machine Learning*, volume 222 of *Proceedings of Machine Learning Research*, pages 518–533. PMLR, 11–14 Nov 2024.
- [27] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. Deepfm: a factorization-machine based neural network for ctr prediction. *arXiv preprint arXiv:1703.04247*, 2017.
- [28] Joel Hestness, Stephen W. Keckler, and David A. Wood. Gpu computing pipeline inefficiencies and optimization opportunities in heterogeneous cpu-gpu processors. In *2015 IEEE International Symposium on Workload Characterization*, pages 87–97, 2015.